CSEN 296B-2: Week 1.2 Submission 1 — Architecture Decision Log

Student Name: Atishay Jain **Date:** April 1, 2026
Project: AstraNotes
Chosen Technical Path: Python

---

1. Architecture Decision Log

Weaker Prompt

> Act as a software architect. Design a backend for AstraNotes that lets users save and organize their notes.

Strengths of the First Response

- It acknowledged the need for a note object and some kind of persistence mechanism.
- It gave a high-level separation between the user-facing layer and storage.

Weaknesses of the First Response

- The response lumped business logic and persistence together without clear boundaries.
- There was no mention of how to handle errors or invalid operations.
- Privacy and data protection for sensitive notes were completely ignored.
- The design did not consider how to test individual components or extend the system later.

Stronger Prompt

> You are a senior Python architect. Design a modular, layered architecture for AstraNotes — a personal note-taking application built in Python. Walk through the design step by step. Cover: (1) the note data model with metadata fields, (2) an abstract repository interface for persistence operations, (3) a file-based storage adapter for saving notes locally, (4) a strategy for handling errors during save, load, and delete operations, (5) how to protect private notes through a dedicated privacy boundary, and (6) how to keep the system testable and extensible so it can later support new note types like voice or image without rewriting existing storage code. Minimize external dependencies and keep the scope realistic for a solo developer working over one academic quarter.

Key Excerpt from the Stronger Response

> **Note Data Model:** Use a Python `dataclass` with fields for `id` (UUID), `title`, `body`, `is_private` (bool), `created_at`, `modified_at`, and `tags`. Each note carries its own identity and metadata from creation.
>
> **Repository Abstraction:** Define a `NoteRepository` abstract base class exposing `save()`, `get()`, `list_all()`, `update()`, and `delete()`. All higher-level logic depends on this interface, never on a concrete storage implementation.
>
> **Local File Adapter:** Implement `JsonFileRepository` as a concrete subclass. Each note is serialized to its own `{id}.json` file in a designated directory. No external database server is required.
>
> **Error Strategy:** Create domain-specific exceptions — `NoteNotFoundError`, `PersistenceError`, `ValidationError` — and catch low-level I/O failures at the adapter boundary so the rest of the application only sees clean, meaningful errors.

> **Privacy Layer:** A standalone `PrivacyService` encrypts the body of any note marked private before it is handed to the repository, and decrypts on retrieval. Using Fernet symmetric encryption keeps this simple and auditable. The privacy logic is independent of storage.
>
> **Extensibility:** New note types (`VoiceNote` , `ImageNote`) subclass the base `Note` model. The repository interface and privacy service work against the base type, so adding a new format does not require changes to the storage or encryption layers.

Refined Prompt

> Update the architecture so AstraNotes starts with text notes but can grow to support voice and image notes without rewriting the persistence layer. List each major component, its single responsibility, and where validation, privacy enforcement, and version tracking belong.

Final Architecture Components

Component	Role
Note Entity	Data model representing a single note (id, title, body, timestamps, tags, privacy flag)
Repository Interface	Abstract contract for CRUD operations, decoupled from any storage technology
JSON File Adapter	Concrete persistence using local JSON files on disk
Privacy Service	Encrypts and decrypts private note bodies outside the storage layer
Validation Layer	Rejects incomplete or malformed notes before they reach persistence
Version History Service	Stores snapshots of prior note states for recovery and auditing
Note Manager (App Service)	Orchestrates validation, privacy, versioning, and storage into a single workflow

Why This Architecture Is Defensible

This architecture is defensible because every component has a single, well-defined job, which makes it straightforward to test each one in isolation and swap implementations when needed. The repository abstraction means the storage backend can change from JSON files to SQLite or a cloud service without rippling through the rest of the codebase. Isolating the privacy service from storage prevents encryption logic from leaking into the UI or persistence layer, reducing the chance of accidental data exposure.

---

2. Architecture Verdict

The refined architecture is a clear improvement over the initial version. It replaces vague, coupled suggestions with explicit component boundaries, defined error contracts, and a realistic extensibility path. The weaker prompt led to a generic answer that would have been hard to implement, test, or explain to a reviewer. The stronger prompt produced a design I can trace directly into backlog items, sprint tasks, and acceptance tests for the rest of the quarter.

---

3. Reflection

The core lesson from this exercise is that prompt quality mirrors design thinking quality. A vague prompt produces a vague design; a specific, constrained prompt produces something you can actually build from. Iterating through three rounds of prompting forced me to articulate what I actually needed from the architecture — modularity, testability, privacy separation — before I ever wrote a line of code.